Unity Catalog Governance

The governance layer that controls access to data and AI assets across every workspace.

01 · 30 MIN
Unity Catalog fundamentals

02 · 40 MIN
The access control model

03 · 25 MIN
Connecting storage securely

04 · 25 MIN
Fine-grained access and lineage

What this session covers

01	**Unity Catalog fundamentals** The object model, the metastore, and how catalogs are structured.	30 min
02	**The access control model** Privileges, inheritance, ownership, admin roles and effective access.	40 min
03	**Connecting storage securely** Access connector, storage credentials, external locations and volumes.	25 min
04	**Fine-grained access and lineage** Row filters, column masks, dynamic views, lineage and audit evidence.	25 min

Session 4 covered who reaches the platform. This session covers what they can see.

By the end, you will be able to

01 Describe the Unity Catalog object hierarchy and the role of the metastore.

02 Explain the privilege model, including inheritance, ownership and the MANAGE privilege.

03 Describe how storage credentials and external locations govern access to ADLS Gen2.

04 Identify the fine-grained controls available and when each is appropriate.

01

Unity Catalog fundamentals

What it governs, and how the objects are arranged.

object model

metastore

catalog patterns

isolation

What is data governance?

The framework of **policy, process and technology** that controls how data is managed, accessed and protected.

Policy

What is allowed, written down, with someone accountable for it.

Process

How access is requested, approved, reviewed and recorded.

Technology

What enforces the policy, and evidences that it was enforced.

> Unity Catalog is the third part. It cannot supply the first two, and this session returns to that at the end.

What happens without it

01 Data spreads across workspaces with no consistent rules, because each team solves access locally.

02 Anyone with access to a table sees every column in it, including the ones they had no business seeing.

03 There is no record of who read what, so a question about exposure cannot be answered at all.

04 Nothing can be demonstrated to a regulator, because there is no artefact to show them.

The fourth is not a technical problem. It is the one that stops a platform going live.

What governance buys

Quality

One definition of a dataset, not several that disagree.

Security

Access matched to role, rather than to whoever asked.

Compliance

Evidence produced as a by-product of normal operation.

Discovery

People find data instead of rebuilding it from source.

Collaboration

Teams share governed data rather than emailing extracts.

The middle three are why it gets approved. The last two are why people stop resenting it.

Governance before Unity Catalog

Every workspace held **its own metastore** — its own table definitions, and its own permissions.

WORKSPACE A	WORKSPACE B	WORKSPACE C
Its own metastore	Its own metastore	Its own metastore
Its own user list	Its own user list	Its own user list
Its own access list	Its own access list	Its own access list

The same dataset registered three times, with three access lists and no way to compare them.

> Nobody could answer "who can see this table?" without checking every workspace by hand.

What Unity Catalog is

One governance layer **above the workspaces** . Assets are defined once, permissions granted once, and every attached workspace sees the same thing.

One place to grant

A privilege is granted once and applies wherever the asset is reached from.

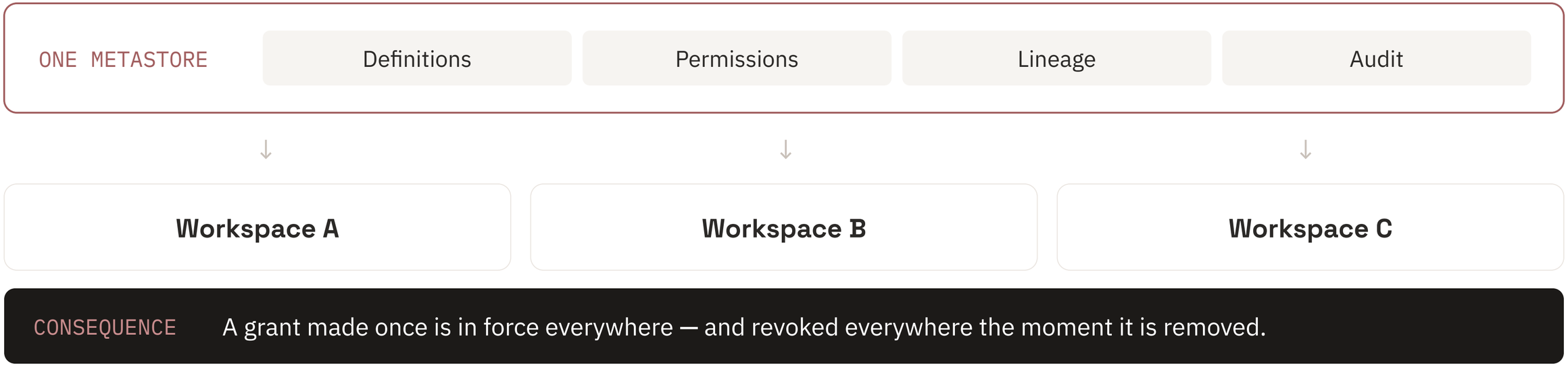
One place to review

Effective access to an object can be listed, rather than reconstructed.

One audit trail

Reads, grants and changes are recorded centrally and attributed to a principal.

One metastore, every workspace



CONSEQUENCE A grant made once is in force everywhere — and revoked everywhere the moment it is removed.

Four pillars

Access control

Who can reach what, granted once and enforced everywhere.

Discovery

Finding the data you are entitled to, with its documentation and tags.

Lineage

Where data came from, and what depends on it downstream.

Auditing

Who did what, and when, recorded without anyone opting in.

Discovery is the pillar security-led discussions forget, and the one that decides whether anyone uses the platform.

The object hierarchy

METASTORE Top-level container for metadata and permissions. One per region.

↓

CATALOG The first part of every object name, and the main unit of isolation.

↓

SCHEMA A grouping within a catalog — usually a subject area or a pipeline layer.

↓

OBJECTS Tables, views, volumes, functions and registered models.

What Unity Catalog governs

Tables and views

Structured data, and the derived objects built over it.

Volumes

Files under governance — the replacement for mount points.

Functions

Registered user-defined functions. A masking function is itself governed.

Registered models

Machine learning models, governed by the same grants as data.

Storage credentials

External locations

Shares for external sharing

The infrastructure that connects to storage is governed the same way. Everything above is a securable object.

One metastore per region

One per region

Created by an account admin, in the region where the data sits.

One per workspace

A workspace attaches to exactly one metastore. Every workspace in the region can share it.

Not shared across regions

Assets in another region's metastore are reached through Delta Sharing, not directly.

For a single-region deployment this is one decision made once. For a multi-region one it shapes everything that follows.

Pattern one: catalog per environment

dev

Broad access, low sensitivity, freedom to create.

test

Same structure, controlled data, used to validate releases.

prod

Narrow access, written only by service principals.

```
PROMOTION   dev.claims.paid_claims → prod.claims.paid_claims
```

Only the first part of the name changes, so the same code runs in every environment.

Pattern two: catalog per domain

`claims`

`policy`

`finance`

`reinsurance`

What it buys

Each catalog has a named owning team who approves access. The platform team stops being the bottleneck.

What it costs

Environment separation has to be expressed somewhere else, usually in the catalog name.

Pattern three: catalog per sensitivity

Where it fits

Where a regulator or internal policy draws a hard line — personal data, or a specific retention or residency rule. The boundary is visible in the name and enforced with one grant.

Where it hurts

It cuts across domains, so one analyst doing one job may need access to two catalogs and join across them.

> Use it for the genuinely restricted tier, not as the primary structure.

Choosing a catalog structure

CATALOG
Environment

→

SCHEMA
Domain

→

INSIDE
Sensitivity, by grant

> Let the catalog boundary match the boundary you most often need to grant or refuse in one move. Everything below it is easier to change later.

Binding a catalog to workspaces

Open

The default. Visible from every workspace on the metastore, subject to privileges.

Bound to workspaces

Visible only from named workspaces. The production catalog cannot be reached from development at all.

Bound read-only

A reporting workspace gets production data with no possibility of a write.

Binding applies regardless of privileges held. It is the control that makes a shared metastore defensible.

02

The access control model

Privileges, inheritance, ownership and the administrative boundaries.

privileges | inheritance | ownership | effective access

Every grant is the same shape

PRINCIPAL

A group, user or service principal

PRIVILEGE

What it may do

SECURABLE

An object named in the namespace

```
GRANT SELECT ON TABLE prod.claims.paid_claims TO `dbx-claims-analyst-prod`;
```

Never to a path on disk, never tied to a cluster. Always a principal, a privilege and a named object.

The privileges you use daily

`USE CATALOG / SCHEMA`	Traversal. Without it, nothing inside the container is reachable at all.
`SELECT`	Read a table, view or volume.
`MODIFY`	Write to it — insert, update, delete.
`CREATE`	Make new objects inside a catalog or schema.

> The most common support call: SELECT granted correctly, USE missing, so the table appears not to exist.

Privileges flow downwards

GRANTED HERE

Catalog

→

APPLIES TO

Every schema in it

→

AND TO

Every object in those schemas

Why it helps

Grant once on a schema, and every table added to it next year is already covered.

Why it bites

A careless grant covers objects that do not exist yet, and nobody re-reviews it when they appear.

Where a grant should sit

Catalog

A whole environment. Rare, deliberate, and approved at a senior level.

Schema

The workhorse. A subject area with a stable audience.

Table

Exceptions only. Each one is something a person has to remember.

> Many table-level grants is a symptom, not a strategy. It means the schema boundary is in the wrong place.

What an owner can always do

An owner needs **no grant at all**. Ownership cannot be revoked — only transferred.

Read and write it

Regardless of what has or has not been granted.

Grant it to others

An owner can extend access without asking anyone.

Drop it

Including a production table, with no further permission.

> Ownership defaults to whoever created the object. Left alone, it is a record of who happened to run the notebook.

Designing ownership

Groups, not people

A group owner survives a leaver, and moves the approval conversation into a team.

The accountable team

For production data, the owner is the team answerable for it — usually not the platform team.

Set at creation

The pipeline sets ownership when it creates the object, rather than an audit correcting it later.

> An individually owned production table is a single point of failure with a person's name on it.

The MANAGE privilege

Administer an object **without owning it** : grant, revoke and change metadata, while ownership stays with the accountable team.

What it is for

Delegated administration — a data steward running access for a domain on the owning team's behalf.

What it is not for

Convenience. It carries the authority to widen access, so it is granted deliberately and reviewed.

Three admin roles

ACCOUNT ADMIN

Creates the metastore

Manages the account, workspaces and identities, and assigns the metastore admin.

METASTORE ADMIN

Top of the governance tree

Can grant on anything in the metastore and reassign ownership of any object.

WORKSPACE ADMIN

No data privileges

Manages entitlements, policies and objects in one workspace. The role grants no access to data.

Platform administration and data access are separate. Being a workspace admin does not let you read the data.

Using the metastore admin role

Assign it to a group

Small, named, and under privileged access management. Never a single person.

Create the top level

Catalogs, external locations and storage credentials — then hand ownership downwards.

Recover orphans

Reassign ownership when an object's owner has left. Only this role can.

Delegate the rest

Routine grants belong to owners and MANAGE holders, not to this group.

If the metastore admin group is doing routine grants, the delegation model has not been set up.

Two ways to grant

Catalog Explorer

Good for exploring what exists, checking effective access, and a genuine one-off change.

SQL GRANT

Text, so it can be reviewed, version-controlled and applied by a pipeline.

Keep the standing access model in the repository. Then every grant has a commit, a reviewer and a history — which is most of what an audit asks for.

One group’s access, in full

```
GRANT USE CATALOG ON CATALOG prod TO `dbx-claims-analyst-prod`; GRANT USE SCHEMA ON SCHEMA prod.claims TO `dbx-claims-analyst-prod`; GRANT SELECT ON SCHEMA prod.claims TO `dbx-claims-analyst-prod`;
```

> Three statements, one group, no table named individually — and tables added to the schema next year are already covered.

Why a revoke can appear to fail

They own it

A revoke does not touch ownership. If the principal is the owner, nothing changes.

It is inherited

Revoking on a table while the schema grant stands has no effect. Remove it at the level it was given.

When removing access, check ownership and inheritance as well as the explicit grant — then verify the result rather than assuming it.

Who can see this table?

Direct grants

Privileges granted on the object itself.

Inherited grants

Anything granted on its schema or its catalog.

Owner and MANAGE

Access that exists without any grant at all. The part people forget.

AT SCALE Catalog Explorer answers it for one object. The information schema answers it for a whole catalog.

What an access review needs

The grants	Queried from the information schema, on a stated date.
The memberships	Who was in each granted group at that date, from Entra ID.
The ownership	Because ownership is access, and it appears in no grant list.

All three on the same date. The grant list alone describes half the access.

The object hierarchy

Which sequence correctly orders Unity Catalog's object hierarchy from top to bottom?

A Catalog, metastore, schema, table

B Schema, catalog, metastore, table

C Metastore, catalog, schema, table

D Metastore, schema, catalog, table

The object hierarchy

C

The metastore sits at the top, holding metadata and permissions; catalogs, then schemas, then securable objects sit below it.

Inheritance and new tables

A group is granted SELECT on the schema prod.gold. Six months later, a new table is added to that schema. Does the group need a new grant to query it?

A Yes, every new table needs its own SELECT grant

B No, the schema-level grant already covers it

C Only if the table is a view, not a managed table

D Only the table's owner can decide, regardless of the schema grant

Inheritance and new tables

B

Privileges flow downwards and cover objects created after the grant. This is also why a careless schema-level grant is dangerous.

Revoking an owner’s access

Dev owns the table prod.gold.sales. An admin runs REVOKE SELECT ON TABLE prod.gold.sales FROM dev. What happens to Dev's access?

A Nothing changes — an owner needs no grant and cannot be revoked this way

B Dev loses read access but keeps write access

C Dev loses all access to the table

D Ownership automatically transfers to the admin

Revoking an owner’s access

A Ownership sits outside the privilege system. It can only be transferred, never revoked by revoking a privilege.

MANAGE vs. ownership

Priya wants Dev to be able to grant and revoke access to a schema on her behalf, but she does not want him able to drop it. What should she give him?

A Ownership of the schema

B Metastore admin

C The MANAGE privilege on the schema

D USE SCHEMA

MANAGE vs. ownership

C

MANAGE separates administering an object — granting and revoking on it — from owning it outright.

Answering ”who can see this table?”

An auditor asks who can currently see prod.gold.customers. Which sources must be checked together to answer correctly?

A Only the direct grants on the table

B Direct grants, group membership behind those grants, and ownership

C Only who currently owns the table

D The storage account's access keys

Answering ”who can see this table?”

B

Grants alone describe half the access. Group membership and ownership both need checking, on the same date, for the full picture.